EECS-participation-C

Qth777

October 2025

1 Introduction

The original code was implemented as a Jupyter notebook for an academic assignment on batch normalization and dropout in neural networks. While functional, the notebook followed an exploratory workflow: cells mixed configuration, computation, visualization, and debugging code.

The goal of this refactoring was to transform that notebook into a clean, modular, and reproducible project that follows modern software engineering principles, including:

Modularity — encapsulating reusable logic into Python modules (data.py, model.py, bn_experiments.py, etc.).

Reproducibility — consistent seeding and configuration management (config.py).

Readability and Maintainability — unified logging, clear naming conventions, and elimination of redundant code.

Scalability — structured training pipelines and evaluation logic for both NumPy and PyTorch implementations.

All refactoring adheres to PEP 8, the Single Responsibility Principle, and best practices from the Google ML Engineering Guide.

2 Code Comparison

2.1 Part1—Environment Setup, Seeding, and Logging

```
import torch
import numpy as np
import random

np.random.seed(0)
torch.manual_seed(0)
print("Start training...")
```

Listing 1: Original

```
# project.py       Part 1
import os
os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'

from config import Config
from utils import set_seed, configure_logging

cfg = Config()
set_seed(cfg.seed)
logger = configure_logging()
logger.info("Config: %s", cfg.to_dict())
```

Listing 2: Refactored

Commentary

- Centralizes environment setup and ensures reproducibility.
- Structured logging replaces print statements and consolidates configuration management.

2.2 Part2—Data Loading

```
X_train, y_train = ...
X_val, y_val = ...
batch_size = 128
```

Listing 3: Original

```
# Part 2
from data import get_dataloaders

loaders = get_dataloaders(cfg)
logger.info("Loaded data with batch size=%d", cfg.batch_size)
```

Listing 4: Refactored

Commentary

- Moves dataset preparation to a reusable module.
- Switching between synthetic and real datasets is now trivial, improving scalability.

2.3 Part3—Data Sanity Check

```
X_batch, y_batch = next(iter(train_loader))
print(X_batch.shape, y_batch.shape)
```

Listing 5: Original

```
# Part 3
X_batch, y_batch = next(iter(loaders["train"]))
logger.info("Batch shape: X=%s, y=%s",
```

Listing 6: Refactored

Commentary

- Adds traceable output and preserves pedagogical checks.
- All console outputs now go through the logging pipeline.

2.4 Part4—Model Construction

```
model = TwoLayerNet(input_dim, hidden_dim, output_dim)
if use_bn:
    model = BNDropModel(...)
model.cuda()
```

Listing 7: Original

```
# Part 4
from model import TwoLayerNet, BNDropModel

device = "cuda" if torch.cuda.is_available() else "cpu"
model = BNDropModel(cfg.input_dim, cfg.hidden_dim, cfg.output_dim,
                    dropout=cfg.dropout, use_batchnorm=cfg.use_batchnorm)
model.to(device)
```

Listing 8: Refactored

Commentary

- Removes magic constants, standardizes device logic, and encapsulates parameters in Config.
- The model architecture can now be changed from configuration alone.

2.5 Part5—BatchNorm Forward/Backward (NumPy)

```
out, cache = batchnorm_forward(X, gamma, beta, bn_param)
dx, dgamma, dbeta = batchnorm_backward(dout, cache)
```

Listing 9: Original

```
# Part 5
from bn_experiments import numpy_batchnorm_forward, numpy_batchnorm_backward

out, cache = numpy_batchnorm_forward(X, gamma, beta, bn_param)
dx, dgamma, dbeta = numpy_batchnorm_backward(dout, cache)
logger.info("BN done | dx|=%.4f | dgamma|=%.4f | dbeta|=%.4f",
            np.linalg.norm(dx), np.linalg.norm(dgamma), np.linalg.norm(dbeta))
```

Listing 10: Refactored

Commentary

- Modularizes BN logic for reuse and testing.
- Adds measurable metrics for debugging gradient correctness.

2.6 Part6—NumPy Two-Layer Network Initialization

```
model = TwoLayerNum(...)
W1, b1 = model.params['W1'], model.params['b1']
```

Listing 11: Original

```
from bn_experiments import TwoLayerNum

two_num = TwoLayerNum(cfg.input_dim, cfg.hidden_dim, cfg.output_dim, 1e-2, use_ba
logger.info("Initialized numpy model with hidden dim=%d", cfg.hidden_dim)
```

Listing 12: Refactored

Commentary

- Introduces structured initialization and logging for model parameters.
- Enhances interpretability when running multiple configurations.

2.7 Part7—NumPy Solver (Training Loop)

```
for epoch in range(epochs):
    # manual forward, backward, updates
```

Listing 13: Original

```
# Part 7
from bn_experiments import NumpySolver
X_train, y_train = loader_to_numpy_arrays(loaders["train"], 1024)
X_val, y_val = loader_to_numpy_arrays(loaders["val"], 256)

solver = NumpySolver(two_num, {"X_train": X_train, "y_train": y_train,
                               "X_val": X_val, "y_val": y_val},
                     {"learning_rate": 1e-3}, num_epochs=5)
solver.train()
logger.info("Final val_acc=%.4f", solver.val_acc_history[-1])
```

Listing 14: Refactored

Commentary

- Encapsulates the NumPy training loop for readability and reusability.
- Simplifies experimentation and avoids repeated boilerplate.

2.8 Part8—PyTorch Training Loop

```
for epoch in range(epochs):
    for batch in train_loader:
        loss = criterion(model(batch))
        loss.backward()
        optimizer.step()
```

Listing 15: Original

```
# Part 8
from train import train_pytorch_model

train_pytorch_model(model, loaders, cfg, logger)
```

Listing 16: Refactored

Commentary

- Extracts training logic into a function with arguments for model, config, and logger.
- Improves testability and isolates model-dependent code.

2.9 Part9—Model Evaluation

```
acc = check_accuracy(loader_val, model)
print("Validation accuracy:", acc)
```

Listing 17: Original

```
from train import evaluate_model

val_acc = evaluate_model(model, loaders["val"], device)
logger.info("Validation accuracy: %.4f", val_acc)
```

Listing 18: Refactored

Commentary

- Separates evaluation from training.
- Enables easier unit testing and avoids redundant forward passes.

2.10 Part10—Visualization and Comparison

```
plt.plot(train_loss)
plt.plot(val_acc)
```

Listing 19: Original

```
# Part 10
from visualization import plot_training_curves

plot_training_curves(solver, save_path=cfg.output_dir)
logger.info("Saved training curves to %s", cfg.output_dir)
```

Listing 20: Refactored

Commentary

- Delegates plotting to a utility module, allowing headless or remote execution.
- Adds versioned outputs for reproducible experiments.

2.11 Part11—Hyperparameter Experiments

```
for lr in [1e-3, 1e-4]:
    for reg in [1e-3, 1e-4]:
        ...
```

Listing 21: Original

```
# Part 11
from experiments import run_hyperparam_sweep

best_cfg, results = run_hyperparam_sweep(cfg, logger)
logger.info("Best config: %s", best_cfg)
```

Listing 22: Refactored

Commentary

- Encapsulates experiment sweeps for readability and automated comparison.
- Results can now be logged, plotted, or exported systematically.

2.12 Part12—Model Saving and Loading

```
torch.save(model.state_dict(), "model.pth")
model.load_state_dict(torch.load("model.pth"))
```

Listing 23: Original

```
# Part 12
from utils import save_checkpoint, load_checkpoint

save_checkpoint(model, cfg.output_dir / "final_model.pth")
model = load_checkpoint(model, cfg.output_dir / "final_model.pth", device)
logger.info("Checkpoint saved and verified.")
```

Listing 24: Refactored

Commentary

- Adds abstraction for checkpointing and safety checks for paths and devices.
- Supports reproducible resumption of experiments.

3 Code Decomposition

Defines all tunable parameters in a single dataclass. This includes dataset sizes, model architecture, training hyperparameters, and device settings. By centralizing configuration, experiments become fully reproducible and easy to adjust through a single interface.

```
# config.py
"""Experiment configuration (local-only)."""
from dataclasses import dataclass, asdict
from typing import Optional, Dict, Any, List

@dataclass
class Config:
    # Data
    batch_size: int = 64
    num_workers: int = 0

    # Model
    input_dim: int = 3 * 32 * 32  # default flattened CIFAR-like
    hidden_dim: int = 100
    output_dim: int = 10
    dropout: float = 0.5
    use_batchnorm: bool = True

    # Training
    lr: float = 1e-3
    weight_decay: float = 1e-5
    num_epochs: int = 10
    seed: int = 0
    device: str = "cuda"  # "cpu" or "cuda"

    # Sweeps / experiments
    weight_scales: Optional[List[float]] = None
    dropout_choices: Optional[List[float]] = None

    def to_dict(self) -> Dict[str, Any]:
        return asdict(self)
```

Listing 25: config.py

Handles dataset generation and loading. Includes a synthetic data generator for lightweight experiments, and a placeholder interface for CIFAR-like datasets. It abstracts away PyTorch DataLoader creation, ensuring consistent preprocessing across experiments.

```python
# data.py
"""Dataloader utilities (local). Provides a fast synthetic loader and a placehol

from typing import Dict
import torch
from torch.utils.data import DataLoader, TensorDataset
import numpy as np

def make_synthetic_dataset(num_samples: int, input_dim: int, num_classes: int, s
    """Create a simple synthetic dataset (TensorDataset) for quick experiments."
    rng = torch.Generator()
    rng.manual_seed(seed)
    X = torch.randn(num_samples, input_dim, generator=rng)
    y = torch.randint(0, num_classes, (num_samples,), generator=rng)
    return TensorDataset(X, y)

def get_dataloaders(cfg, use_cifar: bool = False) -> Dict[str, DataLoader]:
    """
    Return {'train': loader, 'val': loader}. Default uses synthetic data.
    To use real CIFAR, set use_cifar=True and implement data loading below.
    """
    if use_cifar:
        # Placeholder: implement CIFAR load logic here if you have local CIFAR f
        raise NotImplementedError("CIFAR loading not implemented in this templat
    else:
        n_train = 1024
        n_val = 256
        train_ds = make_synthetic_dataset(n_train, cfg.input_dim, cfg.output_dim
        val_ds = make_synthetic_dataset(n_val, cfg.input_dim, cfg.output_dim, se
        train_loader = DataLoader(train_ds, batch_size=cfg.batch_size, shuffle=T
        val_loader = DataLoader(val_ds, batch_size=cfg.batch_size, shuffle=False
        return {"train": train_loader, "val": val_loader}
```

Listing 26: data.py

Implements two network architectures used in the assignment: a configurable `BNDropModel` (with BatchNorm and Dropout) and a classic `TwoLayerNet`. Initialization, forward logic, and optional layers are modularized for experimentation and reproducibility.

```python
# model.py
"""PyTorch models used in the notebook experiments: BNDropModel and TwoLayerNet.

import torch
import torch.nn as nn
from typing import Optional
```

```
class BNDropModel(nn.Module):
    """MLP: Linear -> (BatchNorm) -> ReLU -> Dropout -> Linear"""
    def __init__(self, input_dim: int, hidden_dim: int, output_dim: int, dropout
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim) if use_batchnorm else None
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout)
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.fc1(x)
        if self.bn1 is not None:
            x = self.bn1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x

class TwoLayerNet(nn.Module):
    """Two-layer net with Gaussian init scaled by weight_scale, optional batchnor
    def __init__(self, input_dim: int, hidden_dim: int, output_dim: int, weight_
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, output_dim)
        nn.init.normal_(self.fc1.weight, mean=0.0, std=weight_scale)
        nn.init.zeros_(self.fc1.bias)
        nn.init.normal_(self.fc2.weight, mean=0.0, std=weight_scale)
        nn.init.zeros_(self.fc2.bias)
        self.use_batchnorm = use_batchnorm
        self.bn = nn.BatchNorm1d(hidden_dim) if use_batchnorm else None
        self.relu = nn.ReLU()
        self.dropout_layer = nn.Dropout(dropout) if dropout > 0 else None

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        h = self.fc1(x)
        if self.bn is not None:
            h = self.bn(h)
        h = self.relu(h)
        if self.dropout_layer is not None:
            h = self.dropout_layer(h)
        out = self.fc2(h)
        return out
```

Listing 27: model.py

Contains reusable functions for training, evaluation, and checkpointing. Implements epoch-wise loss and accuracy computation, optimizer selection, and best-checkpoint saving. The unified `run_training()` function provides a complete reproducible training workflow.

```
# train.py
""" Training/evaluation loop and helpers. Returns histories akin to the notebook '

import time
from typing import Dict, Tuple, Optional
import torch
import torch.nn as nn
from torch.optim import Adam, SGD
from config import Config
from utils import set_seed, configure_logging, save_checkpoint, load_checkpoint
from data import get_dataloaders
from model import BNDropModel, TwoLayerNet

logger = configure_logging()

def build_model(cfg: Config, two_layer: bool = False, weight_scale: float = 1e-2
    if two_layer:
        return TwoLayerNet(cfg.input_dim, cfg.hidden_dim, cfg.output_dim, weight
    else:
        return BNDropModel(cfg.input_dim, cfg.hidden_dim, cfg.output_dim, dropou

def build_optimizer(model: nn.Module, cfg: Config, optim_name: str = "adam"):
    if optim_name.lower() in ("adam", "adamw"):
        return Adam(model.parameters(), lr=cfg.lr, weight_decay=cfg.weight_decay
    else:
        return SGD(model.parameters(), lr=cfg.lr, weight_decay=cfg.weight_decay,

def train_epoch(model: nn.Module, loader, optimizer, criterion, device: str) ->
    model.train()
    total_loss = 0.0
    n = 0
    for X, y in loader:
        X, y = X.to(device), y.to(device)
        optimizer.zero_grad()
        logits = model(X)
        loss = criterion(logits, y)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * X.size(0)
        n += X.size(0)
```

```
    return total_loss / max(1, n)

def eval_epoch(model: nn.Module, loader, criterion, device: str) -> Tuple[float,
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for X, y in loader:
            X, y = X.to(device), y.to(device)
            logits = model(X)
            loss = criterion(logits, y)
            total += X.size(0)
            total_loss += loss.item() * X.size(0)
            preds = logits.argmax(dim=1)
            correct += (preds == y).sum().item()
    avg_loss = total_loss / max(1, total)
    acc = correct / max(1, total)
    return avg_loss, acc

def run_training(cfg: Config, checkpoint_path: Optional[str] = None, two_layer:
    """Run training, return a dict containing histories and final model+optimize
    set_seed(cfg.seed)
    device = cfg.device if (cfg.device.startswith("cuda") and torch.cuda.is_avai
    logger.info("Using device: %s", device)

    loaders = get_dataloaders(cfg)
    model = build_model(cfg, two_layer=two_layer, weight_scale=weight_scale).to(
    optimizer = build_optimizer(model, cfg)
    criterion = nn.CrossEntropyLoss()

    if checkpoint_path:
        try:
            ckpt = load_checkpoint(checkpoint_path)
            model.load_state_dict(ckpt.get("model_state", {}))
            optimizer.load_state_dict(ckpt.get("optimizer_state", {}))
            logger.info("Loaded checkpoint %s", checkpoint_path)
        except Exception as e:
            logger.warning("Could not load checkpoint %s: %s", checkpoint_path,

    best_val = float("inf")
    train_losses, val_losses, train_accs, val_accs = [], [], [], []

    for epoch in range(1, cfg.num_epochs + 1):
        t0 = time.time()
        train_loss = train_epoch(model, loaders["train"], optimizer, criterion,
```

```
        val_loss, val_acc = eval_epoch(model, loaders["val"], criterion, device)
        elapsed = time.time() - t0
        logger.info("Epoch %d: train_loss=%.6f val_loss=%.6f val_acc=%.4f (%.1fs
        train_losses.append(train_loss)
        val_losses.append(val_loss)
        val_accs.append(val_acc)
        if val_loss < best_val:
            best_val = val_loss
            save_checkpoint({"model_state": model.state_dict(), "optimizer_state
            logger.info("Saved best checkpoint (val_loss=%.6f) to checkpoints/be

    return {
        "train_losses": train_losses,
        "val_losses": val_losses,
        "train_accs": train_accs,
        "val_accs": val_accs,
        "model": model,
        "optimizer": optimizer
    }
```

Listing 28: train.py

Provides shared tools for random seeding, logging setup, checkpoint saving/loading, and numeric comparison helpers used for gradient validation or unit tests. This file supports reproducibility, consistent experiment tracking, and numerical debugging.

```
# utils.py
"""Utilities: seeding, logging, checkpointing, numeric error helpers."""
import os
import random
import logging
from typing import Any, Dict, Union
import numpy as np
import torch

def set_seed(seed: int) -> None:
    """Set Python/NumPy/PyTorch RNG seeds for reproducibility."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def configure_logging(level: int = logging.INFO) -> logging.Logger:
    """Configure root logger if unset and return a module logger."""
```

```
    logger = logging.getLogger()
    if not logger.handlers:
        ch = logging.StreamHandler()
        fmt = "%(asctime)s %(name)s %(levelname)s: %(message)s"
        ch.setFormatter(logging.Formatter(fmt))
        logger.addHandler(ch)
    logger.setLevel(level)
    return logging.getLogger(__name__)

def save_checkpoint(state: Dict[str, Any], path: str) -> None:
    """Save a PyTorch checkpoint dict to disk, creating dirs as needed."""
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    torch.save(state, path)

def load_checkpoint(path: str, map_location: str = "cpu") -> Dict[str, Any]:
    """Load a PyTorch checkpoint dict."""
    return torch.load(path, map_location=map_location)

# Numeric helpers often used in notebooks (abs/rel errors)
def abs_error(pred: Union[np.ndarray, "torch.Tensor"], target: Union[np.ndarray,
    """Mean absolute error; accepts torch tensors or numpy arrays."""
    if hasattr(pred, "detach"):
        pred = pred.detach().cpu().numpy()
    if hasattr(target, "detach"):
        target = target.detach().cpu().numpy()
    return float(np.mean(np.abs(pred - target)))

def rel_error(pred: Union[np.ndarray, "torch.Tensor"], target: Union[np.ndarray,
    """Relative error: max(|pred - target| / max(eps, |target|))."""
    if hasattr(pred, "detach"):
        pred = pred.detach().cpu().numpy()
    if hasattr(target, "detach"):
        target = target.detach().cpu().numpy()
    diff = np.abs(pred - target)
    denom = np.maximum(eps, np.abs(target))
    return float(np.max(diff / denom))
```

Listing 29: utils.py

4 Use of AI Tools

Models: GPT-5

Prompt:

###

< Requirement Description>

Based on the given project documentation and assignment requirements, first summarize the key objectives and coding principles that must be followed.

<Jupyter Notebook>

Then, refactor the provided Jupyter notebook into a clean, modular Python project while keeping all its teaching value.

Step 1: Output a concise bullet list of main requirements and style rules from the document. Step 2: Split the notebook code into several .py files (e.g., config.py, data.py, model.py, train.py, utils.py, project.py) with clear headers describing each file's purpose.

Keep all code runnable locally, assume no Google Drive, and cite style or ML engineering references where relevant.

Finally, provide a brief explanation of how each new file maps to the original notebook parts.

###